

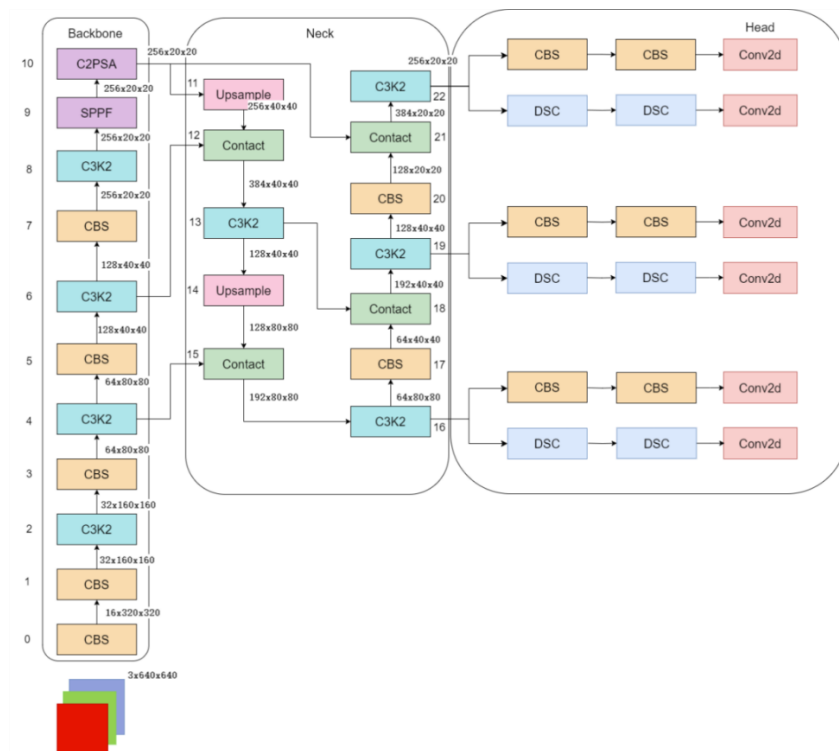
HW2: Long-Tailed Object Detection for Drone-Based Intelligent Counting

1. Model Description

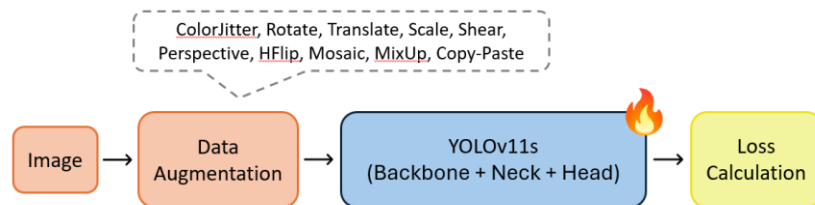
1.1. Core Components & Modifications

The model architecture used is YOLOv11s, loaded from the “yolo11s.yaml” configuration file. This designation indicates a "small" (s) variant of a custom YOLOv11 framework, optimized to strike a balance between speed and accuracy. The model processes the entire image at once by dividing it into a grid and simultaneously predicting bounding boxes, confidence scores, and class probabilities for each grid cell. A significant modification in the training protocol was that the model was trained entirely from scratch. This approach required a robust data augmentation strategy and an extended training schedule to learn the requisite features.

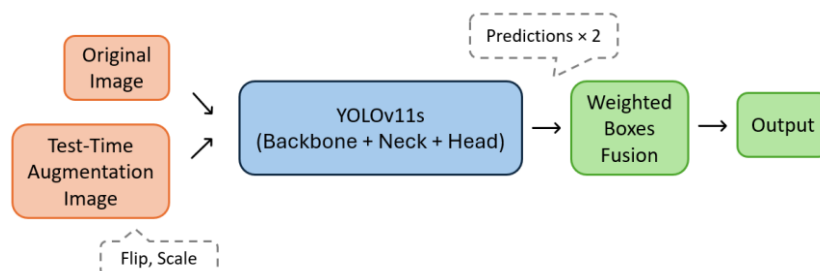
1.2. Architecture Illustration



➤ Training Pipeline



➤ Inference Pipeline



2. Implementation Details

The successful implementation of the model depends on a carefully optimized set of preprocessing steps, augmentations, and training parameters.

2.1. Dataset and Preprocessing

The dataset, defined in “Dataset.yaml”, is divided into training and validation sets with a ratio of 90:10, respectively. All images are resized to a high resolution of 1280×1280 pixels for both training and inference. This high resolution is essential for detecting small objects, which are common in drone-based imagery.

2.2. Data Augmentation

To enhance model generalization and address the dataset's long-tailed distribution, an extensive series of data augmentation techniques was applied to the training set.

Color Jitter: Randomly alters the image's hue, saturation, and value.

Random Rotation: Applied random rotation within a range of ± 15 degrees.

Random Translation: Applied a random translation of up to 10% of the image size.

Random Scaling: Applied random scaling of up to 50%.

Random Shear: Applied random shear transformations within a range of ± 2 degrees.

Random Perspective: Applied slight random perspective transformations.

Random Horizontal Flip: Applied with a 50% probability.

Mosaic: Enabled to combine four training images into a single canvas.

MixUp: Applied with a 15% probability to linearly combine two images and their corresponding labels.

Copy-Paste: Applied with a 50% probability to paste objects onto random backgrounds.

2.3. Hyperparameters

Epochs	Batch Size	Optimizer	Initial Learning Rate	Final Learning Rate Factor	Momentum	Weight Decay
400	4	SGD	0.005	0.01	0.937	0.0005

2.4. Loss Functions

The YOLOv11s model calculates the total loss by combining three primary components:

Classification loss (binary cross-entropy) measures object classification accuracy.

Bounding Box Regression Loss (CIoU Loss) improves localization accuracy.

Distribution Focal Loss (DFL) refines the precision of predicted bounding box coordinates.

The total loss is the weighted sum of these components, with weights explicitly set as: box = 7.5, cls = 1.5, and dfl = 1.5. The mathematical formula is as follows:

$$L_{total} = 7.5 \cdot L_{box} + 1.5 \cdot L_{cls} + 1.5 \cdot L_{dfl}$$

2.5. Training and Evaluation Strategies

The evaluation metric for model selection was the mean Average Precision (mAP) on the validation set, calculated across a range of Intersection over Union (IoU) thresholds from 0.5 to 0.95. The model weights that achieved the highest mAP on the validation set, saved as “best.pt,” were used for final inference.

3. Results Analysis

3.1. Quantitative Improvements Through Multiple Experiments

Step 1. Each experimental configuration was manually adjusted by **changing one or two parameters per experiment** to identify the combination that achieved the highest publicly available score in the Kaggle competition.

Step 2. Record the three loss values on the training set and the mAP on the validation set during each epoch. At the same time, ensure that the total GPU memory usage does not exceed 12 GB.

Step 3. Compare the model results across different configuration combinations.

Experiment	Configuration	Public Score
Case 1	backbone: YOLOv11n, image resize: 640, epochs: 300, batch size: 8, optimizer: SGD, WBF (IoU threshold = 0.6), NMS (IoU threshold = 0.5), max detection: 300, copy-paste: 0.15, cls: 0.5, confidence threshold = 0.001	0.23872
Case 2	backbone: YOLOv11n, image resize: 640, epochs: 300, batch size: 8, optimizer: SGD, WBF (IoU threshold = 0.7), NMS (IoU threshold = 0.6) , max detection: 300, copy-paste: 0.15, cls: 0.5, confidence threshold = 0.001	0.24167
Case 3	backbone: YOLOv11n, image resize: 640, epochs: 400 , batch size: 8, optimizer: SGD, WBF (IoU threshold = 0.7), NMS (IoU threshold = 0.6), max detection: 500 , copy-paste: 0.15, cls: 0.5, confidence threshold = 0.001	0.24373
Case 4	backbone: YOLOv11n, image resize: 1280 , epochs: 400, batch size: 8, optimizer: SGD, WBF (IoU threshold = 0.7), NMS (IoU threshold = 0.6), max detection: 500, copy-paste: 0.15, cls: 0.5, confidence threshold = 0.001	0.25149
Case 5	backbone: YOLOv11s , image resize: 1280, epochs: 400, batch size: 4 , optimizer: SGD, WBF (IoU threshold = 0.7), NMS (IoU threshold = 0.6), max detection: 500, copy-paste: 0.15, cls: 0.5, confidence threshold = 0.001	0.27261
Case 6	backbone: YOLOv11s, image resize: 1280, epochs: 400, batch size: 4, optimizer: SGD, WBF (IoU threshold = 0.7), NMS (IoU threshold = 0.6), max detection: 500, copy-paste: 0.5, cls: 1.5 , confidence threshold = 0.001	0.27472
Case 7	backbone: YOLOv11s, image resize: 1280, epochs: 400, batch size: 4, optimizer: SGD, WBF (IoU threshold = 0.7), NMS (IoU threshold = 0.6), max detection: 800 , copy-paste: 0.5, cls: 1.5, confidence threshold = 0.001, TTA: twice	0.27550

To improve the final predictions on the test set, a multi-stage inference pipeline was developed, incorporating two key techniques:

Test-Time Augmentation (TTA) involves instructing the model to generate predictions not only on the original image but also on augmented versions (e.g., images that have been flipped and rescaled).

Weighted Boxes Fusion (WBF) aggregates all prediction sets generated from repeated TTA. It utilizes a threshold to merge multiple detection boxes, resulting in more robust and accurate final bounding boxes by averaging the coordinates of overlapping boxes based on their confidence scores.

This TTA+WBF approach is computationally more expensive but highly effective in reducing false positives and enhancing the localization accuracy of true positives.

3.2. Solutions to the Long-Tailed Problem

Data-Level: The copy-paste augmentation method directly increases the model's exposure to rare-class instances by pasting them onto diverse backgrounds.

Loss-Level: Increasing the classification loss weight compels the model to prioritize correctly classifying objects, which helps prevent it from neglecting rare classes in favor of the more dominant "head" classes.

Training Strategy: Training from scratch on high-resolution images for an extended period ensures that the model has sufficient capacity and time to learn the subtle features of all classes, including those with few samples.

The experimental results presented in the table indicate that increasing the image resolution from 640 to 1280 and the number of epochs from 300 to 400 improves the mAP score by 0.00982. Furthermore, raising the copy-paste parameter from 0.15 to 0.5 and the classification loss parameter from 0.5 to 1.5 further enhances the mAP score by 0.00211.

3.3. Visualizations Examples

Success Case (Better Prediction)



Failure Case (Worse Prediction)

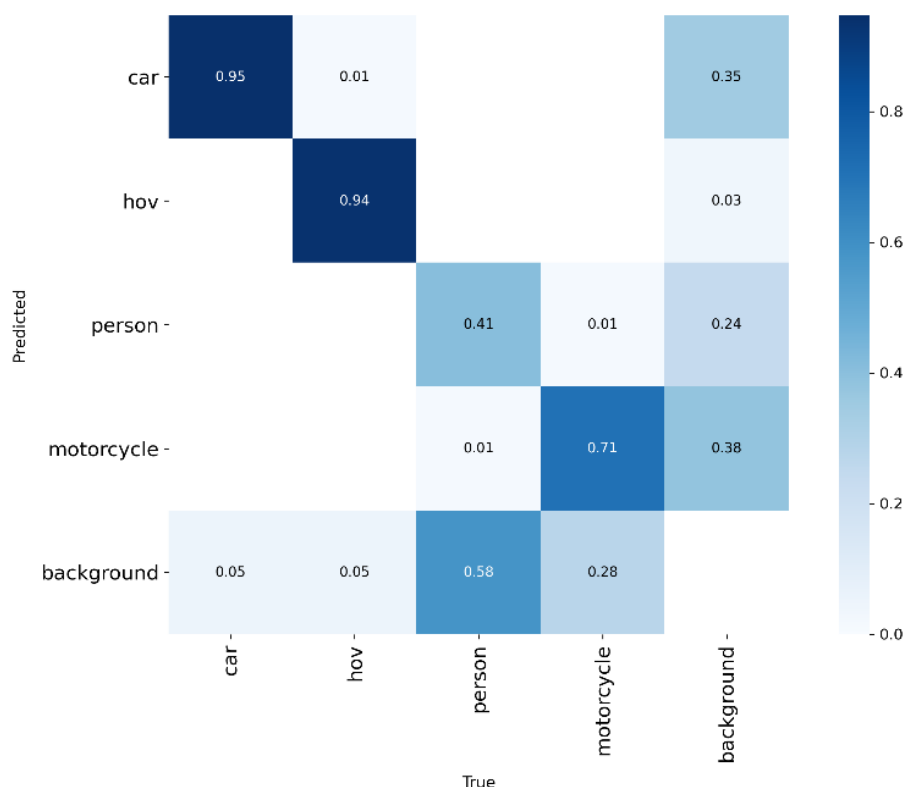


Success Case: High-Density Detection

The model demonstrates high robustness and precision, accurately identifying numerous car instances in a crowded parking lot. The bounding boxes show precise localization, and the confidence scores remain consistently high. Furthermore, the model effectively distinguishes between different classes, correctly identifying both cars and motorcycles within the same scene.

Failure Case: False Positives at Zero Confidence

The image contains a high number of false positives, including cars, motorcycles, and persons detected in implausible locations such as open water and empty roads. Notably, all these false detections have a confidence score of 0.00. This visualization does not indicate a model failure but rather illustrates the raw noise produced by the model.



3.4. Error Analysis of the Normalized Confusion Matrix

Strong Performance: The model demonstrates excellent performance in identifying car and hov classes, correctly classifying 95% of true car instances and 94% of true hov instances.

Moderate Performance: The motorcycle class is identified with reasonable accuracy, with 71% of instances correctly classified.

False Negatives: The person class is clearly the most challenging. A significant 58% of person instances were missed by the model and incorrectly classified as background. Similarly, 28% of motorcycle instances were also missed and classified as background.

False Positives (the final column): Represent predictions that did not match any true object.

4. Short Conclusion

This project successfully implemented a YOLOv11s object detection model for a challenging drone-based, long-tailed counting task. The model was trained from scratch using a highly customized augmentation pipeline to ensure robust feature learning. The final performance was significantly boosted by a sophisticated inference strategy that combined repeated TTA with WBF.